과제#3

- **강의지원시스템에 4/26(일) 23:00까지 제출**
  - 각 과제 폴더명은 1.Deque / 2.StackCalculator 로 하여, 각 폴더에 .java만 두고, 학번.zip으로 압축하여 제출
  - 기존 제출방식과 동일하게 압축을 해제했을 때 폴더 두개가 바로 나오도록 압축할 것

- **1. Deque**
  - 반드시 이중연결리스트에 기반한 class MyDequeu 포함

- **2.StackCalculator**
  - 입력되는 중위 표현의 산술연산을 **1) 후위 표기를 바꾸어 출력하고 2) 계산 결과를 출력**
  - 반드시 class MyStack && (class MyArrayList || class MyQueue) 포함

1.Deque

- **이중연결리스트에 기반한 Deque를 위한 MyDeque 클래스를 설계하고 프로그램하시오.**
  - dummy head 와 dummy tail이 없는 구조로 구현.
  - ADT 설계, 특히 naming에 유의할 것
  - 입출력은 https://www.acmicpc.net/problem/10866 와 동일하게 동작하도록 구현.
  - 당연히! 자바의 ArrayDeque나 LinkedList 사용시 0점 처리
  - 정수 전용 MyDeque가 아닌 generic으로 구현시 추가점수 부여

  - Main.java와 MyDeque.java로 분리해서 구현할 것
    - 설계에서 필요하다면 추가적인 class나 .java 구현 ok

2.StackCalculator

- **1) 중위식을 후위식으로 바꾸어 출력하고**
  - 입출력/문자열 관련 패키지는 “합리적인" 수준에서 사용 허용. 계산식 parsing의 본인 개발 여부는 점수에 반영
    - 계산식 parsing에서 StringBuilder 클래스나, split, StringTokenizer 수준의 메소드 사용은 허용
    - 단, 정규식(regex를 포함) 등의 2학년이 이해할 수 없을 것으로 보이는 메소드 사용시 개별 발표 요구 또는 감점
  - 대상 연산자 : 단항연산자 - ~, 이항 산술 연산자 : * / % + -, 이항 시프트 연산자 << >>,
    이항 비트 연산자 & | ^, 괄호연산자
    - 연산자 우선 순위와 결합법칙은 Java의 원칙에 따름
    - 피연산자들과 연산자들 사이에 항상 공백이 있는 것으로 가정
    - 단항연산자인 – (minus)와 ~ (bitwise not) 도 연산자로 취급하여 구현 할 것
      - 마이너스 단항연산자가 포함된 입력 -1을 “-1”의 정수로 저장하거나, “- - - 1”을 – 개수count 하여 –1으로 축약하는 등은 감점
      - 후위식에서 마이너스(단항연산자)와 빼기(이항연산자)를 구분하기 위해 마이너스를 “m” 등의 기호로 변경 가능
- **2) 계산 결과를 출력: 입력 피연산자와 계산 결과 모두 정수로 구현할 것**
- **단, 계산식의 오류에 대한 적절한 처리할 것**
  - ex. /나 %의 분모가 0. 연산자나 피연산자가 부족한 경우.
  - 오류 메시지는 모두 적절한 영어로 작성할 것(실행 채점시 인코딩 문제)

2.StackCalculator

실행 예제

```
Input(‘Q’ or ‘q’ to quit): 1 + 2 * 4 * ( 5 - 3 + 1 ) + ( 4 + 2 ^ 3 * 2 – 1 )
Postfix: 1 2 4 * 5 3 - 1 + * + 4 2 + 3 2 * 1 - ^ +
Result:  28

Input(‘Q’ or ‘q’ to quit): 11 / 3 * 2 << ( 3 * ( 1 + 7 ) / 5 )
Postfix: 11 3 / 2 * 3 1 7 + * 5 / <<
Result: 96

Input(‘Q’ or ‘q’ to quit): ( 1 - 7 / 2 ) - - ~ - 3 * - - 2
Postfix: 1 7 2 / - 3 m ~ m 2 m m * -
Result: 2

Input(‘Q’ or ‘q’ to quit): 3 * 5 + 4 + * 3
Postfix: 3 5 * 4 + 3 * +
[Error] One or more additional operands are required.

Input(‘Q’ or ‘q’ to quit): 3*5%(4&-2+4>>--2)
Postfix: 3 5 * 4 2 m 4 + 2 m m >> & %
[Error] Divide by zero

Input(‘Q’ or ‘q’ to quit): q
**** QUIT ****
```

단항연산자 –와 ~는
후위연산식으로 변환한 뒤, 계산할 때는
stack에서 피연산자 하나만 pop하여
단항연산을 수행하는 것이 좋은 구조.
마이너스와 빼기가 혼동될 수 있으므로
구분할 수 있는 기호 m으로 변경

“+ *” 가 연산자 연속이라 오류가 아닌
(ex. “2 * – 3 + …” 와 같은 표현이 있
을 수 있으므로), 후위표기 생성 후
계산과정에서 “+” 의 피연산자가 부족해
오류로 인식되는 것이 바람직

공백없는 수식 처리되면 추가 점수

오류 메시지는 영어로 각자 알아서 결정

2.StackCalculator

- **모든 자료구조(MyStack, MyArrayList, MyQueue 등) 반드시 직접 구현**
  - 배열로 구현시 초기 크기는 4로 지정
  - java.util.ArrayList나 java.util.Collections 등 사용 금지
  - 후위식을 String에 저장하면 효율 저하
    - String[] 또는 MyArrayList<String>에 저장하면 중복된 split을 할 필요가 없음
  - generic으로 구현시
    - 후위식 구할 때는 Stack<String>, 계산할 때는 Stack<int>이 필요
    - 후위식은 MyArrayList<Token<String>>이나 MyArrayList<Token> 등에 저장 가능
      - MyQueue<Token<String>> MyQueue<Token> 등도 가능
      - Token을 사용할 경우 Stack은 각각 Stack<Token<String>>, Stack<Token<int>>로 구현 가능
  - 추후 Tree 계산기가 과제로 나갈 예정이므로, 좋은 클래스 설계가 큰 도움이 됨
- **추가 점수 요소**
  - 입력되는 계산식과 변환한 후위식을 MyQueue에 저장하면 추가 점수
  - 피연산자와 연산자간 공백이 없이 동작되면 추가 점수
  - MyStack<> MyArrayList<> MyQueue<>를 generic으로 구현

```
public class Token<T> {
    private T data ;
    private boolean isOP ;
    public boolean isOperator() ;
    …
```

```
public class Token {
    private int value ;
    private String operator ;
    private boolean isOP ;
    private int inStackPriority;
    private boolean isLeftAssociative ;

    Token(String t) {
        operator = t ;
        if ( isOP = isOperator()) {
            inStackProiroty = getInStackPriority() ;
            isLeftAssociative = isLeftAssociation() ;
        }
        else
            value = getIntValue(t) ;
    }
    private int getInStackPriority() {
        switch (data) {
            case '(' : return 9 ; break ;
            case 'm' : return 1 ; break ;
            case '^' : return 2 ; break ;
    ....
```